UnifiedCrawl: Aggregated Common Crawl for Affordable Adaptation of LLMs on Low-Resource Languages

Bethel Melesse Tessema
Ajou University
Suwon, South Korea

Akhil Kedia
Independent Researcher
Seoul, South Korea

Tae-Sun Chung
Ajou University
Suwon, South Korea

arXiv:2411.14343v1 [cs.CL] 21 Nov 2024

Abstract

Large language models (LLMs) underperform on low-resource languages due to limited training data. We present a method to efficiently collect text data for low-resource languages from the entire Common Crawl corpus. Our approach, UnifiedCrawl, filters and extracts common crawl using minimal compute resources, yielding mono-lingual datasets much larger than previously available sources. We demonstrate that leveraging this data to fine-tuning multilingual LLMs via efficient adapter methods (QLoRA) significantly boosts performance on the low-resource language, while minimizing VRAM usage. Our experiments show large improvements in language modeling perplexity and an increase in few-shot prompting scores. Our work and released source code provide an affordable approach to improve LLMs for low-resource languages using consumer hardware. Our source code is available here at `https://github.com/bethelmelesse/unifiedcrawl`.

1 Introduction

Generative AI has become an integral part of our daily lives, assisting us in various ways, whether it is through its Natural Language Processing (NLP) or Computer Vision (CV) specialty.

In the field of Natural Language Processing (NLP), generative models play a pivotal role in generating coherent and contextually relevant text. They leverage deep learning architectures, particularly transformer-based architectures, and are pre-trained on vast amounts of textual data to learn the nuance of language. These models learn the patterns and structures of language from large datasets, allowing them to generate new text similar to the input data.

These models, fueled by Large Language Models (LLMs), are characterized by their extensive size, often measured in terms of the number of parameters, often many billions. This immense number of parameters allows these models to capture complex language patterns and context, resulting in improved performance on various NLP tasks.

For example, OpenAI's GPT (Generative Pre-trained Transformer) series (Brown et al., 2020; OpenAI, 2022, 2023b,a) has played a fundamental role in transforming the public's view and usage of AI NLP tools. GPT-3 (Brown et al., 2020), with its staggering 175 billion parameters, represents a groundbreaking milestone, showcasing the scalability of transformer-based models. This technology has shown a substantial economic potential due to its broad applicability in commercial usage.

1.1 Problem Definition

By leveraging the availability of extensive large and diverse dataset, often composed of high-resource languages, LLMs have shown remarkable performance in generating content within those linguistic contexts, mimicking human-like responses. However, their performance diminishes significantly when prompted with low-resource languages due to the limited availability of training data and resources for those languages. This limitation results in the generation of responses that lack coherence. For example, when prompted with queries in a low-resource language (such as Amharic (ISO:amh), the most widely language in Ethiopia) models like GPT-turbo-

Correspondence: bethelmelesse01@gmail.com

3.5(OpenAI, 2023b) produce incomprehensible outputs. This challenge persists even when inputting prompts in high-resource languages and instructing the model to respond in low-resource languages, resulting in sentences that lack meaningful coherence.

The limitation of LLMs in handling low-resource languages stems from their initial training which heavily relies on vast amounts of primarily English-centered data. Appendices A.1 and A.2 illustrates the distribution of data and the percentage constituting high-resource and low-resource languages in the training process for these LLMs.

Addressing this challenge of adapting LLMs for use in low-resource languages is crucial for democratizing their accessibility and broadening their practical applicability. However, pre-training LLMs can be exceptionally costly, primarily for two main reasons.

First, as mentioned earlier, pre-training LLMs requires an extensive amount of textual data, and low-resource languages often lack the resources to meet this requirement. For instance, in widely used collection Common Crawl (CC) (CommonCrawl, 2007), low-resource languages such as Tagalog, Punjabi, Kurdish, Lao, Amharic, etc., constitute a minuscule fraction (less than 0.01%) compared to other high-resource languages like English, German, and Russian (A).

Second, the resource-intensive nature of training LLMs, characterized by an extensive number of parameters, demands substantial GPU power, memory, and time. For example, models like gpt-3.5-turbo (175 billion parameters), Claude (Bai et al., 2022) (52 billion parameters), and LLaMA (Touvron et al., 2023) (1.6 billion parameters) translate into an exceedingly resource-intensive training process. Table 9 provides the size details of these LLMs. Consequently, the immense size of these LLMs renders training prohibitively expensive and inaccessible to non-wealthy communities/nations, smaller companies, and educational institutions.

In this paper, our primary objectives are to investigate the following research questions:

1. How can we enhance LLMs to perform well in low-resource languages?
2. How can we collect sufficient training data in low-resource languages for LLMs?
3. How can we achieve the above, while being constrained by consumer devices' memory, storage, and compute?

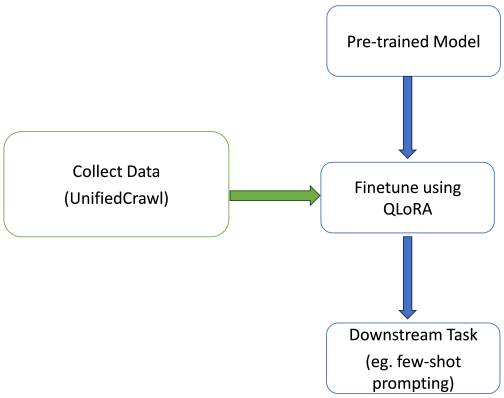

Figure 1: High-Level Overview of our Method

1.2 Proposed Method

To address the aforementioned challenges, we present a novel approach aimed at overcoming data scarcity for low-resource languages, and leverage efficient methods for training LLMs on low-cost hardware.

Our proposed method involves the development of an efficient and cost-effective data collection strategy to extract comprehensive textual content specific to a given low-resource language from the entire Common Crawl corpus. Figure 3 illustrates our architecture. By carefully paying particular attention to memory, compute and network usage in each step of our data collection pipeline, our method is optimized to run entirely on personal consumer hardware - the entirety of the Common Crawl dataset can be achieved in a matter of days, utilizing less than 10GB of RAM and storage. The outcome of this process is our meticulously curated dataset called UnifiedCrawl. Using our method, we were able to successfully extract a monolingual corpora for specific low-resource languages, significantly surpassing the sizes of previously compiled collections, as shown in fig. 2.

Subsequently, we leverage quantization and lightweight low-rank adapters for fine-tuning multilingual Large Language Models (LLMs)

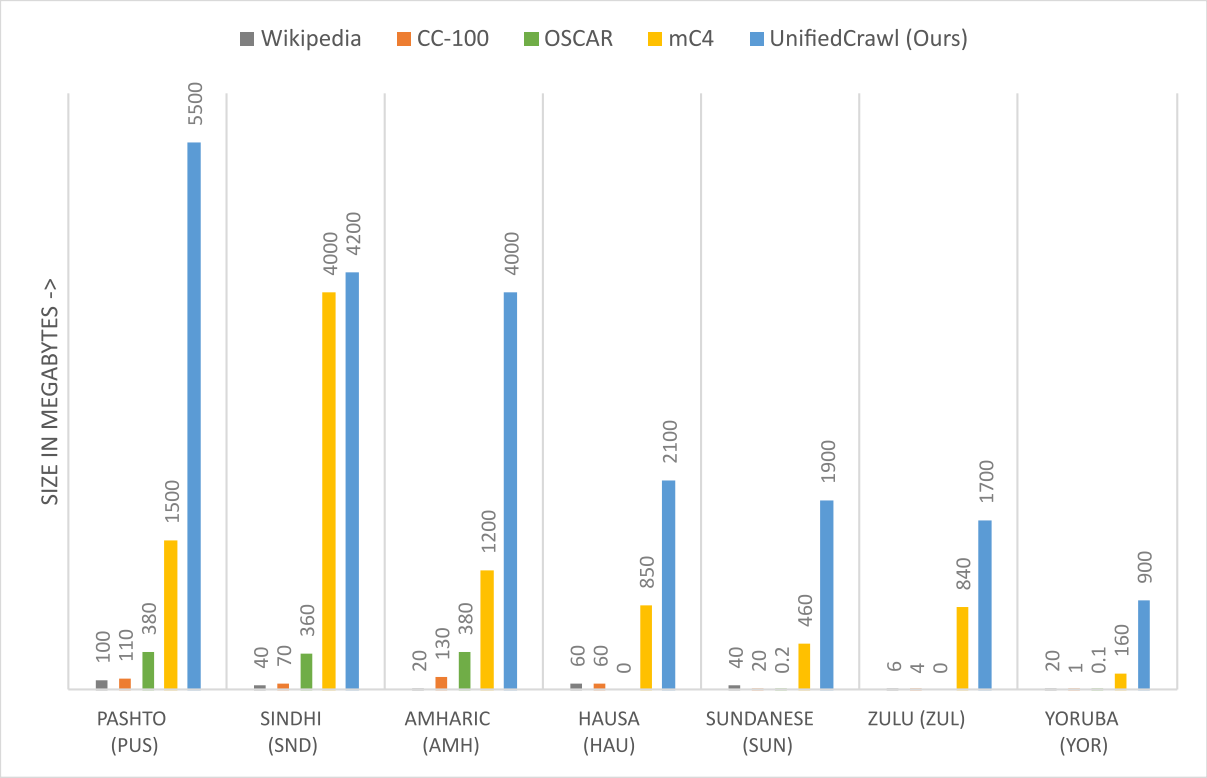

Figure 2: Our Dataset is much Larger than all Prior Works

on the collected dataset. This innovative technique facilitates the utilization of exceptionally large models on consumer-grade GPUs, thereby enhancing accessibility and affordability in training.

Figure 1 illustrates the overarching concept of our proposed scheme. Our approach involves fine-tuning a pre-trained model on our UnifiedCrawl dataset, extracted from the common crawl corpus using our data extraction method. The resulting fine-tuned model can then be applied to downstream tasks.

2 Related Works

2.1 Multilingual Large Language Models

In recent years, there has been a notable surge in the development of multilingual Large Language Models (LLMs), contributing to improved cross-lingual understanding. Examples of these large language models are shown in the appendix B, including their model type, size and the number of languages they are trained on.

While these multilingual models have made strides towards linguistic inclusivity by covering over many languages, they still overlook hundreds of lower-resource languages with sizable speaker populations. This hinders the efficacy of models in many languages compared to performance in high-resource languages. This limitation is primarily due to the lack of sufficient online training data available for lower-resource languages.

In this work, we aim to improve the performance of the above-mentioned models (particularly XGLM model) in low-resource languages by training them on our collected dataset.

2.2 Large Multilingual or Monolingual Datasets

We have noted that data is a crucial component for training language models specifically in the multilingual domain. However, there is a considerable gap in data quantity across languages and domains. Even within the largest Common Crawl corpus, which is a vast web archive that encompasses a diverse collection of web pages providing a rich source of textual data in a multitude of languages and topics, over 41 languages make up less than 0.01% of the data, and 100 languages 0.1% - The quantity of data in common crawl decreases almost

exponentially as shown in figs. 4 and 5. This leaves only a handful of the world's languages represented in evolving language technologies and applications (Joshi et al., 2020).

In this study, we extract all the available textual data from every archive within the common crawl for a specific low resource language. Our choice of the common crawl dataset is driven by our objective to maximize the acquisition of the available data, leveraging its vast size and inclusivity of many languages, as it systematically scraps data across the internet.

2.3 Common Crawl and Dataset Extraction

Due to the extensive scope, Common Crawl (CC) and its derivatives are often used to pre-train large language models, and the majority of State-of-the-art models, such as LLaMA (Touvron et al., 2023) , GPT-3 (Brown et al., 2020) , Falcon (Almazrouei et al., 2023) , PALM (Chowdhery et al., 2023) , Stable LM (Islamovic, 2023) , etc. have incorporated datasets sourced from the Common Crawl corpus into their training pipelines. This integration has contributed to their improved proficiency in understanding and generating human-like text across various domains and languages.

Several smaller datasets have been extracted from the Common Crawl corpus, each contributing to the training of language models. Examples include CC-net (Wenzek et al., 2020), which extracted monolingual corpora for transformer model training; mC4 (AllenAI, 2021), which collected data from publicly accessible CC archives; and OSCAR project (Abadji et al., 2022), which focuses on releasing monolingual corpora from recent CC archives. These subsets have then been used to train many of the State-of-the-art models, such as mT5 (used mC4) (Xue et al., 2021), BLOOM (used OSCAR) (Scao et al., 2022), etc.

However, a common issue persists: many extracted datasets from the common corpus are often limited to one language (eg. CC-net), or a few archives(eg. OSCAR), or are not updated with latest common crawl dumps (eg. mC4). Moreover, due to the sheer scale of the corpus, naively extracting text data for a specific language from all the common crawl archives can be challenging, as it can be time and memory intensive. Moreover, these datasets can also not be easily updated with more data from latest common crawl archives. This limitation hinders the extraction of data for specific languages, especially for very low-resource languages, contributing to the lack of linguistic diversity in the available datasets.

In response to these challenges and limitations, we present a cost-effective means of extracting text data from all CC archives for low-resource languages, including the latest common crawl archives which are much larger compared to previous archives. Our contribution includes releasing the code base for other fellow researchers to extract their own dataset from CC for low resource languages. By doing so, we aim to address the existing gaps in dataset extraction methodologies and contribute to the advancement of linguistic research in low-resource language contexts.

2.4 Deduplication

Another method adopted in our work includes deduplication techniques. Raw text datasets obtained through web scraping often contain the same lines multiple times (Lee et al., 2022). This repetition within the dataset can negatively affect the learning process as it slows down the training as well as limits the model's generalization capabilities. To overcome these challenges, it is important to apply some form of deduplication on the extracted dataset.

Numerous deduplication methods have been previously proposed and employed in prior works. CC-Net, for instances, utilized a paragraph-based Exact-Hash deduplication, whereas other approximate methods, such as MinHash (Broder, 1997), MinHash-LSH (Baluja and Covell, 2007), SimHash (Sadowski and Levin, 2007; Gyawali et al., 2020), are sometimes used for faster deduplication in different context (Scao et al., 2022; Almazrouei et al., 2023).

In our data extraction pipeline, we opted for (Lee et al., 2022)'s exact substring deduplication method – same approach adopted in mC4, OSCAR, and CC100. This approach not only effectively addresses redundancy but also removes common header/footer artifacts often present in the extracted text, enhancing

the overall quality of the dataset. By employing this deduplication method within our proposed scheme, our goal is to extract a high-quality dataset that contributes positively to model's training, resulting in accelerated training, improved perplexity, and reduced likelihood of model memorization.

2.5 Low Resource Model Adaptation

Training (pretraining/fine-tuning) large language models is often impractical beyond major corporate or institutional settings due to their substantial parameter count and the resource-intensive nature of training LLMs. For instance, training a model with a large number of parameters consumes considerable GPU memory and time - for example a 7B model requires 28GB of GPU VRAM, outside the scope of most consumer GPUs. An effective solution to mitigate this challenge involves the integration of quantization techniques into LLMs. Quantization can be achieved through methods such as Quantization-aware training (Wang et al., 2023) or post-training quantization approaches like GptQ (Frantar et al., 2023), SmoothQuant (Xiao et al., 2023), bitsandbytes (Dettmers et al., 2022), among others. These techniques work by reducing the precision of model parameters, allowing for more efficient storage and computation, dramatically reducing GPU VRAM usage.

However, fine-tuning still demands expensive gradients over model parameters. To address this, a more resource-efficient approach involves training adapters on frozen LLMs, known as Low-Rank adaptation (LoRA), proposed by (Hu et al., 2022). LoRA strategically freezes pre-trained model weights and introduces a smaller trainable weight into the model's architecture. As only these added low-rank matrices are trained, there is significant reduction of the overall trainable parameter count and optimizer states, and correspondingly of GPU memory requirement. Experiments on several pretrained models, such as RoBERTa (Liu et al., 2019), DeBERTa (He et al., 2021), GPT-2 (Radford et al., 2019), GPT-3 (Brown et al., 2020), have shown that LoRA achieves comparable or even better performances than existing adapter and prompt-based methods.

This method was further extended to QLoRA (Dettmers et al., 2023), which combines quantization with adapter training. QLoRA achieves a further reduction in memory usage, enabling the fine-tuning of a 65-billion-parameter model on a single 48GB GPU while maintaining full 16-bit fine-tuning task performance.

As QLoRA consistently delivers performance similar to full fine-tuning (Lu et al., 2023; Luo et al., 2023; Manvi et al., 2023; Liu et al., 2023) for much lower VRAM, we adopt QLoRA in our work to balance computational efficiency with model performance.

3 Methods

In this section, we first present a method and procedure to collect and process training data for low-resource languages from the common crawl dataset using limited computing resources. Additionally, we adopt a method to efficiently train large language models on the extracted training dataset using limited GPU resources.

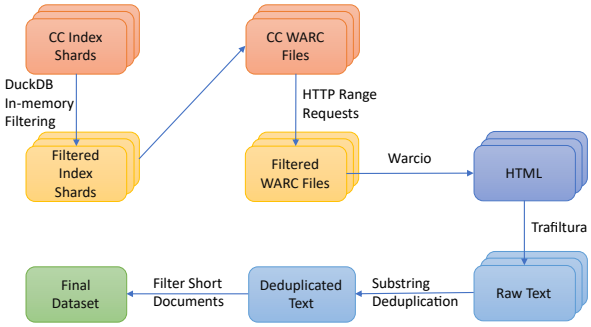

Figure 3: UnifiedCrawl: Data Extraction Framework

3.1 Data Collection Framework

The raw text data for low-resource languages is gathered from the common crawl dataset. The common crawl dataset is extremely large, at approximately 100 TeraBytes per crawl archive, with multiple archives per year[1]. Due to its sheer size, it is difficult to directly download raw text dataset from the corpus. In this subsection, we propose an efficient and cost-effective framework to extract data from a single common crawl archive, which is repeated for all available dumps. Figure 3 illustrates our

[1] https://commoncrawl.org/blog/jan-feb-2023-crawl-archive-now-available

data collection pipeline for extracting raw text dataset from a single common crawl archive.

3.1.1 Index Filtering

The Common Crawl provides a columnar index (CC index[2]) containing language annotations[3] for each URL in the archive. We exploit this information to selectively extract a specific low-resource language from a single archive (eg. CC-MAIN-2023-23). However, even this CC index is typically 100s of GBs for a single archive. As we process 43 archives, this would result in a total of 10s of TBs of just the index, which would be difficult to store.

Instead, we utilize DuckDB (Raasveldt and Mühleisen, 2019), an in-memory analytical database management system, to filter the index shards corresponding to our target language as the primary content language. By employing DuckDB's in-memory filtering and downloading, we eliminate the need for storage-intensive initial bulk downloads and subsequent filtering.

Additionally, we integrate python's multiprocessing package with DuckDB to further reduce the overall downloading time. This integration utilizes multiple processes concurrently across all CPU cores on a single system, leveraging parallelism and bypassing the global interpreter lock to expedite the data acquisition process. The combined utilization of DuckDB and multiprocessing significantly optimizes storage usage and accelerates the overall downloading process.

3.1.2 Extracting WARC Files

The filtered and downloaded index shards from the previous step contains the path to the WARC (Web ARChive) files (Kunze et al., 2008), which contains the content of the crawled webpages and its metadata. Due to the considerable size of the CC archive shards containing these WARC files, downloading all WARC files is impractical. Instead, we selectively filter and retain WARC files corresponding to our target languages within the index shards – avoiding the download of all the WARC files.

This filtering and downloading process utilizes the columnar information provided in the index shard that include the WARC filename, WARC record offset, and WARC record length (the URL and Content Languages we leveraged earlier during the index filtering step are also present here). The WARC filename gives a URL to the CC archive shard containing this particular WARC file, the WARC record offset indicates the exact location of the WARC file we need, the WARC length specifies its span.

Leveraging this information, we download the corresponding WARC for each URL containing our target language as its primary content language via HTTP Range Requests (Fielding et al., 2014). This method of downloading allows us to download only the WARC files that we need and skip the rest from downloading. This is done by requesting the server to send only a portion of an HTTP message back to the client – in our case, only the files corresponding to our target language. By only downloading these necessary WARC files, we conserve bandwidth and storage that result from downloading all the WARC files.

3.1.3 Text Extraction

The next step involves extracting the raw text. We start by retrieving the HTML source from the downloaded WARC files by using the WARCIO library (Contributors, 2017), which offers a fast and efficient way to read and write WARC format. From this extracted html source, we finally obtain the raw text using the command line tool Trafilatura (Barbaresi, 2021) (same tool as used in (Penedo et al., 2023)), which is specifically designed to extract text from the web. This library not only facilitates the extraction of the text from HTML but also improves the text quality by eliminating the noise caused by recurring elements such as headers, footers and other information.

It is noteworthy that the entire process, including the downloading of WARC files as well as the reading and the text extraction, is conducted in-memory for the purpose of reducing the time and storage requirement. By avoiding storing unnecessary elements found within the warc files and the raw html inside the warc (such as large quantities of javascript/html tags/etc), we dramatically reduce the storage requirements.

[2] https://commoncrawl.org/blog/index-to-warc-files-and-urls-in-columnar-format

[3] https://commoncrawl.org/blog/august-2018-crawl-archive-now-available

3.1.4 Deduplication

While the Trafilatura library improves the quality of our extracted text, it is common to have some repetitive sequences within the raw text data. These repetitive sequences include things like copyright notices, some headers/-footers, keywords, etc. that are common in many similar websites across its pages. Having multiple repetitive sequence reduces the overall quality of the training data as it encourages the model to prioritize memorization rather than make the model learn to generalize. Therefore, to further improve the quality of our data as well as improve the training process (later on), we remove these deduplicated elements from the documents. We adopted exact substring deduplication (Lee et al., 2022) technique to remove all duplicate occurrences over a given length from the dataset. However, after removing the repeated sequences, some documents become too short – we hence discard documents of very short length. This final step yields to our final dataset, which we call UnifiedCrawl.

3.2 Low Resource Model Adaptation

Common multi-lingual Large Language Models (LLMs) often suffer from low performance on the long-tail of low-resource languages((Lin et al., 2022), and table 4). Finetuning LLMs on our dataset offers a pathway to democratize AI, improving LLMs on low-resource languages. We hence focus on the regime of using consumer hardware.

LLMs require large GPU memory to simply store the parameters, limiting the maximum model size we can train/infer. Using 4-bit quantization (Dettmers et al., 2022), we can fit almost 3-4x larger models in the same GPU memory, compared to using 16-bit parameters, at the cost of some precision. The performance improvements of a 4x larger LLM compared to a smaller one 16-bit precision is much larger than the slight loss caused by reduced precision, hence it is beneficial to use as large a model as we show in section 5.2.

Furthermore, finetuning LLMs would also require large GPU memory to store gradients and optimizer states for all parameters. We instead leverage Quantized Low-Rank Adapters (QLoRA, (Dettmers et al., 2023)) to efficiently train adapters on the quantized LLMs. This significantly reduces memory usage without compromising performance, enabling training of much larger models. Using QLoRA with larger models outperforms full finetuning of smaller models as we will show in section 6.1.

Our data extraction method results in datasets larger than prior datasets sourced from the common crawl. We also show that our model adaptation method results in large improvements on Language Modeling perplexity, and on few-shot prompting (Brown et al., 2020) on downstream Question-Answering tasks (section 5.2.2) and outperforms full-finetuning of smaller models.

4 Experimental Settings and Implementation Details

4.1 Languages and Benchmark Datasets and Dataset Collection

4.1.1 Dataset Collection

Data collection of UnifiedCrawl was carried out using consumer-grade 500MBps internet connection. Our extracted raw text was formatted in the HuggingFace (Wolf et al., 2020) dataset format. Since the substring deduplication method of (Lee et al., 2022) cannot directly handle datasets in this format, we utilized Text-dedup (Kocetkov et al., 2023) to wrap (Lee et al., 2022)'s implementation of deduplication for compatibility with HuggingFace format. We removed all duplicate substrings of length at-least 50 and all documents shorter than 100 characters, a decision made arbitrarily but following the approach of (Penedo et al., 2023).

4.1.2 Compute Requirements

Index filtering is constrained by network download bandwidth for any low-resource language, as the majority of the index is discarded, filtering down to 10s of MBs. The index for all the archives can be processed in a few days on a consumer internet connection of 500MBps using $<$ 10GB of RAM. Alternatively, using a single AWS server with 12Gbps network, each archive can be processed in $<$ 20 minutes, and the entire CC filtered for $<$ 4USD in $<$ 1 day. Cloud big-data querying services

such as AWS Athena[4] can run this step much faster, but at the cost of 100s of USD. Text extraction and de-duplication from an archive can be processed in only a few minutes, and all CC archives can be processed in a few hours.

4.1.3 Languages

Our data extraction method underwent testing on seven languages: Hausa (hau), Pashto (pus), Amharic (amh), Yoruba (yor), Sudanese (sun), Sindhi (snd), and Zulu (zul), ordered by the descending number of speakers. We specifically selected very low-resource languages (that constituted less than 0.004% of the Common Crawl dataset each), with the highest number of speakers. Table 1 provides details on each language's ISO code, corresponding number of speakers (in millions), their representation percentage in the Common Crawl dataset (for the ' CC-MAIN-2023-14" archive), and the geographical region where these languages are spoken. By applying our method to these languages, we aim to demonstrate that our implementation and approach are language-agnostic.

4.1.4 Benchmark Datasets

To assess the scale and efficacy of our UnifiedCrawl dataset, we conducted comparative analysis of its size against other notable datasets sourced from common crawl. The dataset included in this benchmarking are OSCAR, mC4, CC-100 and Wikipedia. This comparative evaluation aims to provide insights into the relative size and representativeness of Unifiedcrawl in comparison to these widely used datasets.

4.2 Models and Model Adaptation Settings

4.2.1 Models

Given the language expertise of the authors of this thesis, particularly in Amharic, we focused our model adaptation and evaluation on datasets in this specific language. Following the extraction of UnifiedCrawl-Amharic from the Common Crawl Corpus using our method, we fine-tuned a multilingual large language model using the lightweight adapter QLoRA. Among the available pre-trained multilingual large language models, we chose the XGLM model (Lin et al., 2022) for adaptation. This XGLM Model is available in two size variants: 564M and 4.5B models.

This choice of model is due to its inclusion of the language Amharic in its pretraining dataset. However, this requirement only applies to the XGLM-4.5B parameter model. The XGLM-564M does not include Amharic in its training data. However, we still explored the adaptation process on the smaller model as well. This deliberate selection enables us to explore and analyze the nuances of the adaptation process, considering the variations in language inclusion within the same model. Furthermore, XGLM is larger in size than mGPT, and it performs equally or better than BLOOM (Yong et al., 2023).

4.2.2 Model Adaptation

We use the HuggingFace (Wolf et al., 2020) library to implement our code base. We use $r = 2$ for LoRA rank, as(Hu et al., 2022) found small values of r to be effective, and train adapters on all Linear matrices. We finetune these models on our UnifiedCrawl-Amharic dataset for 1 epoch. While multiple epochs should yield better performance, we only train for 1 epoch due to compute constraints. We used original/standard hyper-params wherever applicable and did grid search for learning rate. All experiments were carried out on an Nvidia RTX3070 or a RTX3090, and finetuning took 1 day.

4.3 Evaluation Settings

Our model was evaluated under two settings – in terms of language modeling, and downstream few-shot prompting.

4.3.1 Language Modeling Evaluation

For evaluating the model's capabilities, we compare the perplexity of our model during fine-tuning using QLoRA (Dettmers et al., 2023) on our UnifiedCrawl-Amharic dataset and the original XGLM model, for both variants. Perplexity is defined as the exponential of negative cross-entropy of language modeling, and is a measure of how well our language model is doing in predicting the next word in a sequence given the previous ones. Lower perplexity implies the model is becoming better at predicting the next word in a se-

[4] https://aws.amazon.com/athena/

Language (ISO)	Fraction of CC	# Speakers(M)	Geographical Region
Hausa (hau)	0.0036%	80	Nigeria, Chad, Cameroon, Ghana
Pashto (pus)	0.0033%	60	Afghanistan, Pakistan
Amharic (amh)	0.0036%	60	Ethiopia
Yoruba (yor)	0.0011%	50	Benin, Nigeria, Togo
Sundanese (sun)	0.0011%	40	Indonesia
Sindhi (snd)	0.0017%	30	Pakistan, India
Zulu (zul)	0.0016%	30	South Africa, Lesotho

Table 1: Details of 7 languages used for Data Collection Evaluation

quence. Perplexity provides a quantitative and direct measure for comparing different models.

4.3.2 Downstream Evaluation

Testing the language model on downstream tasks is necessary to evaluate the model's practical applicability, generalization capabilities, and task-specific performance in real-world scenarios. We test our finetuned model on our UnifiedCrawl-Amharic dataset on a downstream task – in order to evaluate the effectiveness of the fine-tuning process. It helps us to know whether our model has learned useful representations during the fine-tuning process, and that it can be applied to diverse tasks beyond language modeling task, including those it wasn't explicitly trained on.

4.3.3 Question Answering Task

We chose question Answering, which is a task of generating a response given a question and a context, for evaluating our method's performance on downstream application. QA tasks are valuable downstream evaluations for pre-trained language models as they assess the model's comprehension, reasoning abilities, and contextual understanding. Therefore, by evaluating on QA tasks, we can evaluate how well a language model can extract and synthesize information from the context provided, infer relationships between different parts of the text, and generate coherent responses for a given query. We use the AmQA dataset (Abedissa et al., 2023) for evaluating the model performance on a downstream Question-Answering task.

4.3.4 Few Shot Prompting Evaluation

This downstream Question-Answering was done under the few-shot prompting (Brown et al., 2020) setting, where the model is given only a small set of examples and is expected to generate the output. This is to assess whether our model can generalize and adapt quickly to new or unseen scenarios, given limited information. For few-shot evaluation on AmQA test set, we use 10 random Context-Question-Answer examples in the prompt. This number was chosen because more examples in the prompt will simply get truncated due to sequence length limitations. We chose these examples from the AmQA train set ,and we appended a question and context to this prompt chosen from the test sample. Our aim is to generate an answer for the question that is selected from the test set. The closer the answer generated to the ground truth label the better. This few-shot evaluation roughly takes 30 minutes for 4.5B XGLM model.

4.3.5 Evaluation Metrics

We used F1 and EM(Exact Match) scores to evaluate the overall quality and accuracy of our model, as commonly used in Question Answering tasks. F1, which is the harmonic mean of precision and recall, provides a more nuanced evaluation, considering the partial overlaps between the generated answers and the ground truth. Complementing F1 score, the EM score gives the percentage of prediction that exactly matches the ground truth answer. We provided a detailed performance assessment in the next chapter.

5 Performance Evaluation

We present analysis of the UnifiedCrawl-language dataset extracted using our data extraction pipeline. We then show experimental results and analysis of the XGLM models fine-tuned on UnifiedCrawl-Amharic using QLoRA. We evaluate the adapted models based on language modeling perplexity and downstream few-shot prompting performance on Question-Answering on AmQA.

5.1 Data Collection Evaluation

We processed a total of 43 archives, starting from "CC-MAIN-2018-43", which marks the first archive to have language annotations[5]. Using our proposed data collection approach, we collected a monolingual dataset for 7 languages. This includes Hausa (hau), Pashto (pus), Amharic (amh), Yoruba (yor), Sundanese (sun), Sindhi (snd) and Zulu (zul), chosen based on the number of speakers vs latest crawl size.

In the following subsection, we provide a detailed analysis focused on the Amharic (amh) language. The final dataset sizes extracted from the Common Crawl for all seven languages are presented in table 2.

5.1.1 UnifiedCrawl Amharic

Index Filtering: Amharic (ISO: amh) is approximately 0.0036% of Common crawl[6]. Each Common Crawl archive index is $\approx$ 250GB compressed. Hence, the expected size of filtered index should be $0.0036\% * 250GB \approx$ 10MB (single archive percentage in the CC * size of archive index). The index filter process resulted in $\approx$ 20MB of filtered index uncompressed, as expected. We only keep URLs with the only language as our target language to increase the dataset quality as well as speed up the process. Keeping URLs with any occurrence increases the size of the filtered index by $3x$.

Extracting WARC files: Each archive has $\approx$ 100TB of compressed WARC files. We only download WARCs corresponding to the target language using Range requests, downloading $\approx$ 3.5GB WARC per archive.

Final Text Extraction: Extracting plaintext from the WARC HTML reduces the size down to 90MB, yielding our final total dataset size of 4GB for all archives.

Deduplication: Sub-string deduplication is first performed within each archive, and then across all archives. Within each archive, deduplication reduces the size by 60% to 40MB, 1.8GB across all archives. This is de-duplicated to provide our final dataset of size 600MB. Combined, the two de-duplication reduced the dataset size by 85%.

5.1.2 UnifiedCrawl for other Languages

Similarly, we provide the final sizes of our UnifiedCrawl datasets across 7 languages in table 2. The first column indicates the languages for which we extracted the datasets, the second column provides the size of the datasets extracted with the primary language exclusively in the content (e.g., content_language=[amh]), and the third column estimates the size of datasets where the primary language is our target language but also includes some minor content from other languages (e.g., content_language=[amh, en,..]). Allowing pages with minor content in other languages should increase the dataset size significantly, and we verified this for Yoruba (yor). The size for other languages are estimated based on the fraction of URLs containing other minor languages.

Languages (ISO)	Size	Max Size
Hausa (hau)	2.1	7
Pashto (pus)	5.5	20
Amharic (amh)	4.0	24
Yoruba (yor)	0.9	2
Sundanese (sun)	1.9	6
Sindhi (snd)	4.2	15
Zulu (zul)	1.7	6

Table 2: UnifiedCrawl-Language Dataset Size. The Size and Max Size are in GBs

5.1.3 Dataset Comparison with other Datasets

Using our method, we were able to extract monolingual corpora that exceeds the size of other prior art for low-resource languages, often by multiple orders of magnitude.

[5] https://commoncrawl.github.io/cc-crawl-statistics/plots/languages

[6] https://commoncrawl.github.io/cc-crawl-statistics/plots/languages

For example, our extracted dataset (UnifiedCrawl-Amharic) surpasses the sizes of previous datasets for the Amharic language. To illustrate, Amharic Wikipedia dataset is 22MB[7], the Amharic News Corpus (Azime and Mohammed, 2021) is 150MB, OSCAR (Abadji et al., 2022) is 500MB, and mC4 (AllenAI, 2021) is 1.2GB. In contrast, our dataset amounts to 4GB before the deduplication step.

Similarly, we show comparison of the size of our UnifiedCrawl-Language dataset to other prominent datasets, OSCAR[8], mC4[9] , CC-100[10], and Wikipedia[11] in table 3. All sizes in this table are in MB. OSCAR, mC4, CC-100 are datasets sourced from the Common Crawl Corpus, whereas Wikipedia dataset is a collection of cleaned articles of all languages built from the Wikipedia dumps[12] using Tensorflow Datasets.

5.2 Method Evaluation

We evaluate the performance of the models fine-tuned using QLoRA models on our UnifiedCrawl dataset in two settings – first, we compare their language modeling capability measured through perplexity (PPL) in upstream, and second, we evaluated the model on downstream few-shot prompting tasks. For both cases, we take the original model as a baseline.

5.2.1 Language Modeling Evaluation

For evaluating pre-training performance in upstream, we analyze the model's perplexity (PPL) during the training process to measure its language modeling capability.

We present the results in table 4, where models marked as "ours" are fine-tuned on UnifiedCrawl-Amharic dataset using QLoRA. Both our fine-tuned models using QLoRA, XGLM-564M and XGLM-4.5B exhibit significantly lower perplexity to that of the original XGLM models.

Models	PPL
XGLM-564M	14,974.70
XGLM-564M (ours)	**105.5**
XGLM-4.5B	35.6
XGLM-4.5B (ours)	**19.6**

Table 4: Language Modeling Evaluation on Amharic

The original XGLM-564M model had a PPL of 14,974.7, as it was not trained on Amharic. The perplexity was dramatically lowered to 105.6 when trained on our UnifiedCrawl-Amharic dataset. Similarly, the PPL for the XGLM-4.5B model decreased from 35.6 to 19.6, indicating a 45% improvement.

These results demonstrate fine-tuning the models using QLoRA on our dataset can lead to significant reductions in perplexity for across model sizes.

5.2.2 Downstream Few Shot Prompting

In downstream tasks, we compare the original model with the fine-tuned QLoRA model on the Amharic dataset under few-shot prompting. We report the F1 score and the EM (Exact Match) score for these evaluations.

Few-shot performance comparisons between the original and fine-tuned models are shown in table 5, where models marked as "ours" are fine-tuned on UnifiedCrawl-Amharic dataset using QLoRA. Since the XGLM-564M was not pre-trained on Amharic, both F1 scores and EM scores are 0, and the score remained unchanged even after fine-tuning this model on our UnifiedCrawl-Amharic. The model is too small, and trained for too few tokens to perform reasonably for few-shot prompting.

However, for the XGLM-4.5B model, the F1 score increased by 24% from 8.0 to 9.9 after fine-tuning, and the EM score increased from 1.3 to 2.3.

This demonstrates fine-tuning specifically benefited the larger model, boosting its few-shot prompting performance on Question-Answering.

[7]Amharic Wikipedia at TF datasets: https://www.tensorflow.org/datasets/catalog/wikipedia#wikipedia20230601am

[8]OSCAR Dataset Size: https://huggingface.co/datasets/oscar

[9]mC4 Dataset Size: https://github.com/allenai/allennlp/discussions/5265

[10]CC-100 Dataset Size: https://data.statmt.org/cc-100/

[11]Wikipedia Dataset Size: https://www.tensorflow.org/datasets/catalog/wikipedia

[12]https://dumps.wikimedia.org/

Languages (ISO)	OSCAR	mC4	CC-100	Wikipedia	UnifiedCrawl
Hausa (hau)	-	850	60	60	**2100**
Pashto (pus)	380	1500	110	100	**5500**
Amharic (amh)	380	1200	130	20	**4000**
Yoruba (yor)	0.1	160	1	20	**900**
Sundanese (sun)	0.2	460	20	40	**1900**
Sindhi (snd)	360	4000	70	40	**4200**
Zulu (zul)	-	840	4	6	**1700**

Table 3: Size of UnifiedCrawl-Language vs. Prior Works

Models	F1	EM
XGLM-564M	0	0
XGLM-564M (ours)	0	0
XGLM-4.5B	8.0	1.3
XGLM-4.5B (ours)	**9.9**	**2.3**

Table 5: Few-shot Prompting Score on AmQA.

6 Ablation Studies

We conduct ablation studies to analyze the impact of different modeling choices and validate the effectiveness of our approach. Specifically, we compare using full fine-tuning versus only adapting with lightweight QLoRA modules, examine trade offs between leveraging pre-trained versus randomly initialized models, and evaluate whether gains from pre-training on our UnifiedCrawl corpus translate to improved performance on downstream tasks.

6.1 Comparison with Full Finetuning

We compared the LM perplexity of our model, where we only trained a lightweight adapter QLoRA while keeping the original parameters frozen, to a fully fine-tuned model, where all the parameters of the model are trained without any adapters. We used our UnifiedCrawl-Amharic Dataset to train these models, in both cases.

The results of this comparison is shown in table 6, where LM PPL is reported on Unified-Crawl-Amharic, and Few-shot F1/EM are on AmQA. Due to memory constraints on our GPU, we only performed full fine-tuning on the XGLM-564M model, as attempts to fully train a 4.5B parameter model resulted in out-of-memory (OOM) errors.

We observe that fully fine-tuning the 564M parameter model yields slightly better language modeling perplexity (76.7 vs. 105.6) compared to using QLoRA to train adapters. However, full fine-tuning requires significantly more GPU memory and computational resources, i.e., it costs higher VRAM and compute compared to using QLoRA, for a relatively minor improvement.

Furthermore, full-finetuning dramatically under-performs compared to using QLoRA on a larger model for the same compute - for example, the 564M model achieves 76.7 PPL with full-fine-tuning, whereas the 4.5B model achieves 19.6

We also evaluated the performance in a downstream few-shot prompting setting on the smaller model. We observed that the Few-shot prompting scores remained zero for both cases (i.e., Full Fine-tuning vs QLoRA) for the 564M model, whereas the 4.5B QLoRA model achieves F1 score of 9.9. This further highlights the importance of using larger models with QLoRA.

6.2 Comparison with Training from Scratch

We also compared using QLoRA to adapt pre-trained models, to training a new model from scratch. For fair comparison, we use the same compute budget for all models as required for training the XGLM-4.5B model for 1 epoch on UnifiedCrawl-Amharic. We train a "base" sized model with 110M params, as well as a 74M model ("compute-optimal" model size, based on Chincilla (Hoffmann et al., 2022) for our compute constraints).

The results are shown in table 7, where LM PPL is reported on Unified-Crawl-Amharic, and Few-shot F1/EM are on AmQA.

Model	LM PPL	Few-shot F1	Few-shot EM
XGLM-564M (full finetune)	**76.7**	0	0
XGLM-564M (ours)	105.6	0	0
XGLM-4.5B (full finetune)	OOM	-	-
XGLM-4.5B (ours)	**19.6**	**9.9**	**2.3**

Table 6: Comparison of QLoRA with Full-fine-tuning

We observed that at equal compute, our model(trained by using QLoRA) shows a substantial performance improvement compared to models that are trained from scratch (without QLoRA). From this, we conclude that training an already pre-trained models by using adapters are better than training a model from scratch, as these models can effectively utilize their prior knowledge gained from multilingual pre-training.

6.3 Comparison on Downstream Supervised Training

We also compare our models (fine-tuned on UnifiedCrawl-Amharic) with baseline models on downstream supervised learning on QA task (AmQA). We use QLoRA for all the models, as training 4.5B model results in OOM.

We present these results in table 8, where PPL, F1 and EM are on the AmQA dataset. Models marked here with "QLoRA" are the original pre-trained models, which we fine-tune on downstream task using QLoRA. Models marked as "ours" add an additional step of fine-tuning on UnifiedCrawl-Amharic before the downstream training.

While the 564M model shows improvements in all scores, the perplexity of both the 4.5B models, the baseline model and the model trained on UnifiedCrawl-Amharic, are very comparable, and so is their F1 and EM score.

The gains observed in Language Modeling and in Few-shot Prompting did not translate to gains on downstream supervised training. While the PPL of the XGLM-564M improved from 99.4 to 59.5, the PPL for the XGLM-4.5 model fine-tuned on the UnifiedCrawl-Amharic remained the same to that of the original model. This could perhaps be due to limited size or quality of this downstream dataset, which is only 1600 training samples from very few wikipedia articles.

Models	PPL	F1	EM
XGLM-564M(QLoRA)	99.4	0.6	0.2
XGLM-564M (ours)	**59.2**	**2.9**	**0.7**
XGLM-4.5B (QLoRA)	2.2	35.0	20.5
XGLM-4.5B (ours)	2.2	34.7	20

Table 8: Supervised-Training Score on AmQA.

7 Limitations and Future Works

While our data extraction approach proves effective for low-resource languages, its applicability to high-resource languages is constrained by prolonged extraction time and storage challenges due to their abundance. Visualization reveals that conventional evaluation metrics, such as F1 and EM, may not adequately capture the nuances in the relationship between ground truth and predicted answers, given the linguistic diversity across languages.

As a future research direction, the data collection pipeline could be expanded on additional low-resource languages beyond the ones mentioned in this work. Additionally, our approach can be enhanced to improve the quality and diversity of the extracted data. We also believe exploring alternative model architectures, such as BLOOM and mT5, during the fine-tuning stage holds promise for achieving enhanced practical deployment. Moreover, a more comprehensive evaluation across diverse downstream tasks is essential to validate real-world performance gains resulting from our extracted data, UnifiedCrawl, and the recommended model adaptation technique.

By addressing these research directions, we aim to develop a robust technique that effectively broadens the accessibility and capabilities of Large Language Models (LLMs) for low-resource languages. This approach contributes to the global democratization of Natural Lan-

Model	LM PPL	Few-shot F1	Few-shot EM
GPT2-74M (scratch)	105.2	1.2	0
GPT2-110M (scratch)	106.1	1.3	0
XGLM-4.5B (Ours)	**19.6**	**9.9**	**2.3**

Table 7: Comparison of QLoRA with training from scratch

guage Processing (NLP) by making advanced language models more widely available and applicable.

8 Conclusion

To summarize, our key contributions are twofold: First, we introduced an efficient technique to aggregate and extract large monolingual datasets for low-resource languages from the entire Common Crawl corpus. By selectively filtering archived data and minimizing storage needs, we obtained raw text data larger than any existing sources using only consumer hardware. Second, we demonstrated effective adaptation of multilingual LLMs by fine-tuning lightweight adapter modules on our extracted datasets. Fine-tuning 4.5B parameter models with adapters using QLoRA resulted in significant perplexity reductions and gains in few-shot prompting scores on Amharic language, with less than 1 GPU-day of compute. Our method and source code make progress towards democratizing LLMs.

References

Julien Abadji, Pedro Ortiz Suarez, Laurent Romary, and Benoît Sagot. 2022. Towards a cleaner document-oriented multilingual crawled corpus. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 4344–4355, Marseille, France. European Language Resources Association.

Tilahun Abedissa, Ricardo Usbeck, and Yaregal Assabie. 2023. Amqa: Amharic question answering dataset. *ArXiv preprint*, abs/2303.03290.

AllenAI. 2021. The C4 Multilingual Dataset · allenai/allennlp · Discussion 5265 - github.com. `https://github.com/allenai/allennlp/discussions/5265`. [Accessed 15-12-2023].

Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. 2023. The falcon series of open language models. *ArXiv preprint*, abs/2311.16867.

Israel Abebe Azime and Nebil Mohammed. 2021. An amharic news text classification dataset.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosiute, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemí Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: harmlessness from AI feedback. *ArXiv preprint*, abs/2212.08073.

Shumeet Baluja and Michele Covell. 2007. Audio fingerprinting: Combining computer vision & data stream processing. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing, ICASSP 2007, Honolulu, Hawaii, USA, April 15-20, 2007*, pages 213–216. IEEE.

Adrien Barbaresi. 2021. Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pages 122–131, Online. Association for Computational Linguistics.

Andrei Z. Broder. 1997. On the resemblance and containment of documents. In *Compression and Complexity of SEQUENCES 1997, Positano, Amalfitan Coast, Salerno, Italy, June 11-13, 1997, Proceedings*, pages 21–29. IEEE.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. Palm: Scaling language modeling with pathways. *J. Mach. Learn. Res.*, 24:240:1–240:113.

CommonCrawl. 2007. Common Crawl - Open Repository of Web Crawl Data - commoncrawl.org. `https://commoncrawl.org/`. [Accessed 15-12-2023].

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

Warcio Contributors. 2017. GitHub - webrecorder/warcio: Streaming WARC/ARC library for fast web archive IO - github.com. `https://github.com/webrecorder/warcio`. [Accessed 15-12-2023].

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Roy T. Fielding, Yves Lafon, and Julian Reschke. 2014. Hypertext Transfer Protocol (HTTP/1.1): Range Requests. RFC 7233.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. OPTQ: accurate quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Bikash Gyawali, Lucas Anastasiou, and Petr Knoth. 2020. Deduplication of scholarly documents using locality sensitive hashing and word embeddings. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 901–910, Marseille, France. European Language Resources Association.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. Deberta: decoding-enhanced bert with disentangled attention. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. 2022. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

Anel Islamovic. 2023. Introducing Stable LM Zephyr 3B: A New Addition to Stable LM, Bringing Powerful LLM Assistants to Edge Devices — Stability AI - stability.ai. https://stability.ai/news/stablelm-zephyr-3b-stability-llm. [Accessed 15-12-2023].

Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. 2020. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 6282–6293, Online. Association for Computational Linguistics.

Denis Kocetkov, Raymond Li, Loubna Ben allal, Jia LI, Chenghao Mou, Yacine Jernite, Margaret Mitchell, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro Von Werra, and Harm de Vries. 2023. The stack: 3 TB of permissively licensed source code. *Transactions on Machine Learning Research*.

John A. Kunze, Gordon Mohr, and Michael Stack. 2008. The WARC File Format (Version 0.16). Internet-Draft draft-kunze-warc-00, Internet Engineering Task Force. Work in Progress.

Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8424–8445, Dublin, Ireland. Association for Computational Linguistics.

Xi Victoria Lin, Todor Mihaylov, Mikel Artetxe, Tianlu Wang, Shuohui Chen, Daniel Simig, Myle Ott, Naman Goyal, Shruti Bhosale, Jingfei Du, Ramakanth Pasunuru, Sam Shleifer, Punit Singh Koura, Vishrav Chaudhary, Brian O'Horo, Jeff Wang, Luke Zettlemoyer, Zornitsa Kozareva, Mona Diab, Veselin Stoyanov, and Xian Li. 2022. Few-shot learning with multilingual generative language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 9019–9052, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Xiao-Yang Liu, Guoxuan Wang, and Daochen Zha. 2023. Fingpt: Democratizing internet-scale data for financial large language models. *ArXiv preprint*, abs/2307.10485.

Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike Lewis, and Luke Zettlemoyer. 2020. Multilingual denoising pre-training for neural machine translation. *Transactions of the Association for Computational Linguistics*, 8:726–742.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized BERT pretraining approach. *ArXiv preprint*, abs/1907.11692.

Yadong Lu, Chunyuan Li, Haotian Liu, Jianwei Yang, Jianfeng Gao, and Yelong Shen. 2023. An empirical study of scaling instruct-tuned large multimodal models. *ArXiv preprint*, abs/2309.09958.

Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, and Wei Lin. 2023. Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. *ArXiv preprint*, abs/2310.08975.

Rohin Manvi, Samar Khanna, Gengchen Mai, Marshall Burke, David B. Lobell, and Stefano Ermon. 2023. Geollm: Extracting geospatial knowledge from large language models. *ArXiv preprint*, abs/2310.06213.

Niklas Muennighoff, Thomas Wang, Lintang Sutawika, Adam Roberts, Stella Biderman, Teven Le Scao, M Saiful Bari, Sheng Shen, Zheng Xin Yong, Hailey Schoelkopf, Xiangru Tang, Dragomir Radev, Alham Fikri Aji, Khalid Almubarak, Samuel Albanie, Zaid Alyafeai, Albert Webson, Edward Raff, and Colin Raffel. 2023. Crosslingual generalization through multitask finetuning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15991–16111, Toronto, Canada. Association for Computational Linguistics.

OpenAI. 2022. Introducing ChatGPT - openai.com. https://openai.com/blog/chatgpt. [Accessed 15-12-2023].

OpenAI. 2023a. Gpt-4 technical report.

OpenAI. 2023b. Introducing ChatGPT and Whisper APIs - openai.com. https://openai.com/blog/introducing-chatgpt-and-whisper-apis. [Accessed 15-12-2023].

Barun Patra, Saksham Singhal, Shaohan Huang, Zewen Chi, Li Dong, Furu Wei, Vishrav Chaudhary, and Xia Song. 2023. Beyond English-centric bitexts for better multilingual language representation learning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15354–15373, Toronto, Canada. Association for Computational Linguistics.

Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The refinedweb dataset for falcon LLM: outperforming curated corpora with web data, and web data only. *ArXiv preprint*, abs/2306.01116.

Mark Raasveldt and Hannes Mühleisen. 2019. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, pages 1981–1984. ACM.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Caitlin Sadowski and Greg Levin. 2007. Simhash: Hash-based similarity detection.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilic, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Villanova del Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, Lucile Saulnier, Samson Tan, Pedro Ortiz Suarez, Victor Sanh, Hugo Laurençon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa, Alham Fikri Aji, Amit Alfassy, Anna Rogers, Ariel Kreisberg Nitzav, Canwen Xu, Chenghao Mou, Chris Emezue, Christopher Klamm, Colin Leong, Daniel van Strien, David Ifeoluwa Adelani, and et al. 2022. BLOOM: A 176b-parameter open-access multilingual language model. *ArXiv preprint*, abs/2211.05100.

Zhixing Tan, Xiangwen Zhang, Shuo Wang, and Yang Liu. 2022. MSP: Multi-stage prompting for making pre-trained language models better translators. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6131–6142, Dublin, Ireland. Association for Computational Linguistics.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *ArXiv preprint*, abs/2302.13971.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. Bitnet: Scaling 1-bit transformers for large language models. *ArXiv preprint*, abs/2310.11453.

Guillaume Wenzek, Marie-Anne Lachaux, Alexis Conneau, Vishrav Chaudhary, Francisco Guzmán, Armand Joulin, and Edouard Grave. 2020. CCNet: Extracting high quality monolingual datasets from web crawl data. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4003–4012, Marseille, France. European Language Resources Association.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 38087–38099. PMLR.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498, Online. Association for Computational Linguistics.

Zheng Xin Yong, Hailey Schoelkopf, Niklas Muennighoff, Alham Fikri Aji, David Ifeoluwa Adelani, Khalid Almubarak, M Saiful Bari, Lintang Sutawika, Jungo Kasai, Ahmed Baruwa, Genta Winata, Stella Biderman, Edward Raff, Dragomir Radev, and Vassilina Nikoulina. 2023. BLOOM+1: Adding language support to BLOOM for zero-shot prompting. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11682–11703, Toronto, Canada. Association for Computational Linguistics.

A Distribution of Languages in Common Crawl

A.1 Distribution of Languages in Common Crawl except English

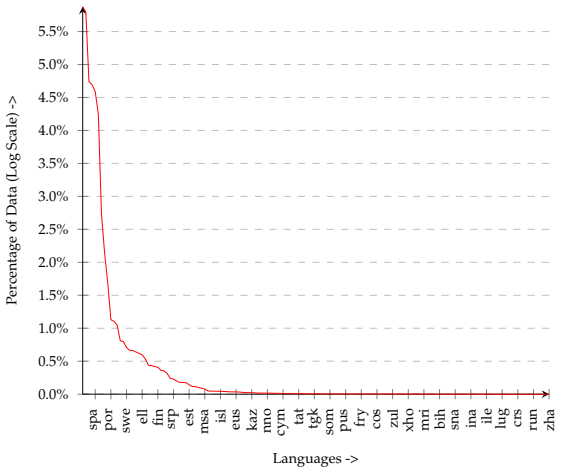

Figure 4: Distribution of Languages in Common Crawl except English

A.2 Distribution of Languages in Common Crawl after Top 60

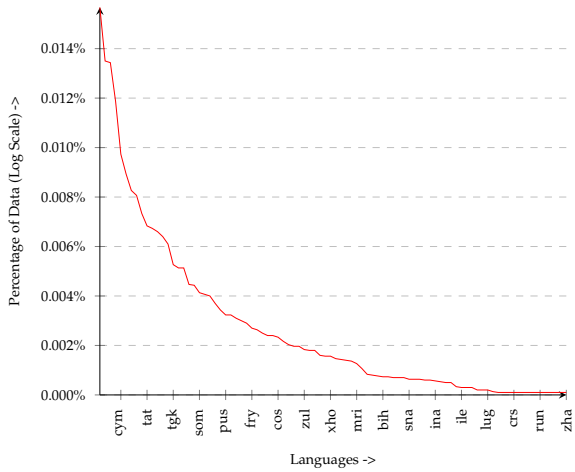

Figure 5: Distribution of Languages in Common Crawl after Top 60

B Overview of Multilingual LLMs

See table 9

Model Type	Multilingual LLMs	Size (# Params)	# Languages
Encoder-Only	mBERT (Devlin et al., 2019)	180M	104
	XLM-R (Conneau et al., 2020)	225M-10.7B	15/100
	XY-LENT (Patra et al., 2023)	480M-2.1B	21
Decoder-Only	XGLM (Lin et al., 2022)	540M-7.5B	30/134
	mGPT (Tan et al., 2022)	1.3B	101
	PaLM (Chowdhery et al., 2023)	540B	122
	BLOOM (Scao et al., 2022)	560M-175B	46
	BLOOMZ (Muennighoff et al., 2023)	560M-175B	46
	GPT-3 (Brown et al., 2020)	175B	1
Encoder-Decoder	mT5 (Xue et al., 2021)	580M-13B	101
	mT0 (Muennighoff et al., 2023)	580M-13B	101
	mBART (Liu et al., 2020)	680M	25

Table 9: Overview of Multilingual LLMs